

auth-service — Documentación

****Puerto:**** 8081

****Base de datos:**** `auth\_db` (PostgreSQL)

****Responsabilidad:**** Gestión de usuarios, autenticación y emisión de JWT

Estructura de archivos

...

auth-service/src/main/java/com/spendiq/auth/

- |— AuthApplication.java → Punto de entrada del servicio
- |— config/
 - | — SecurityConfig.java → Configuración Spring Security + BCrypt
- |— controller/
 - | — AuthController.java → Endpoints REST (/auth/**)
- |— dto/
 - | — RegisterRequest.java → Datos que manda el frontend al registrarse
 - | — LoginRequest.java → Email + password al hacer login
 - | — AuthResponse.java → Token JWT + datos usuario que devuelve el servidor
- |— entity/
 - | — UserEntity.java → Tabla "users" en PostgreSQL
- |— repository/
 - | — UserRepository.java → Acceso a la BD (JPA + Hibernate)
- |— service/
 - | — AuthService.java → Lógica de negocio

...

Endpoints

Método	Ruta	Protegida	Descripción
---	---	---	---
POST	`/auth/register`	No	Registrar nuevo usuario
POST	`/auth/login`	No	Iniciar sesión
GET	`/auth/me`	Sí (JWT)	Obtener perfil del usuario
PUT	`/auth/me`	Sí (JWT)	Actualizar perfil

| POST | `/auth/reset-password` | No | Recuperar contraseña |

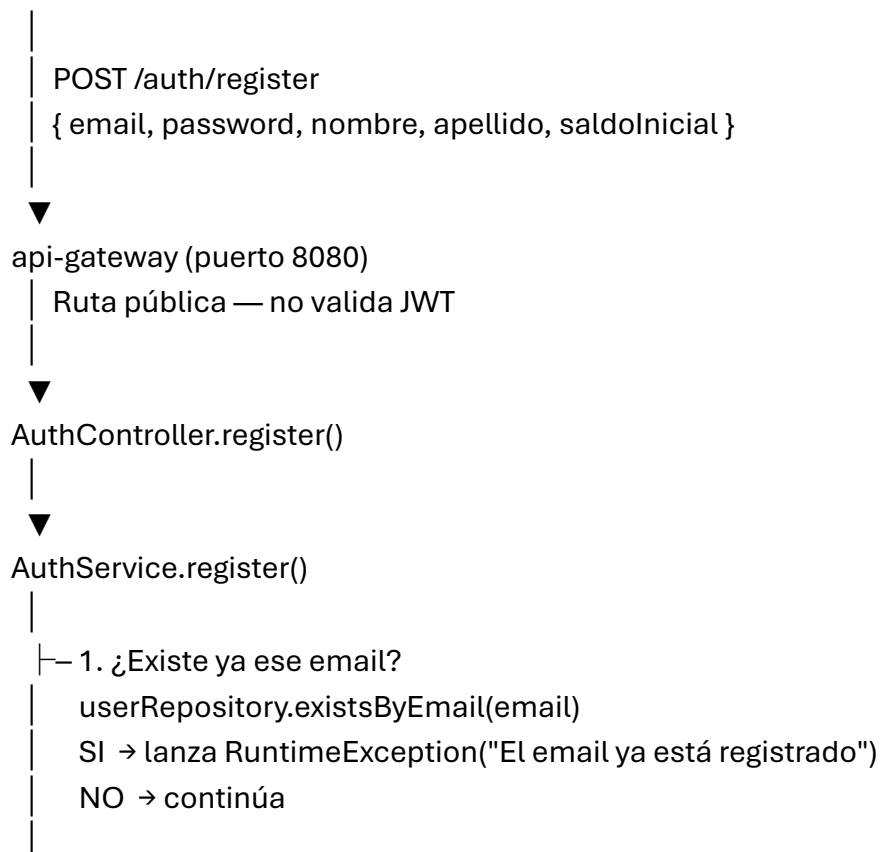
Modelo de datos — tabla `users`

Campo	Tipo	Restricciones	Descripción
id	VARCHAR (UUID)	PK, auto-generado	Identificador único
email	VARCHAR	UNIQUE, NOT NULL	Identificador de login
password	VARCHAR	NOT NULL	Hash BCrypt — nunca texto plano
nombre	VARCHAR	NOT NULL	Nombre del usuario
apellido	VARCHAR	—	Apellido (opcional)
saldoInicial	DOUBLE	—	Balance de partida (default 0.0)
rol	VARCHAR	NOT NULL	USER o ADMIN (default USER)

Flujo de Registro

`, `

Frontend



- └─ 2. Construye UserEntity con los datos del RegisterRequest
- └─ 3. Cifra la contraseña con BCrypt
 - passwordEncoder.encode(password)
 - "miPassword123" → "\$2a\$10\$Xk9..." (hash irreversible)
- └─ 4. Guarda en la BD
 - userRepository.save(user)
 - Hibernate genera: INSERT INTO users (...)
- └─ 5. Genera el JWT con generateToken(user)
 - Contiene: userId, email, nombre, rol
 - Expira en 24 horas
- └─ 6. Devuelve AuthResponse
 - { token: "eyJ...", user: { id, email, nombre, ... } }
 - (la contraseña NO viaja — @JsonIgnore en el campo)

Frontend

- └─ Guarda el token en localStorage

...

Flujo de Login

...

Frontend

- POST /auth/login
- { email, password }



api-gateway → AuthController.login() → AuthService.login()

- └─ 1. Busca usuario por email
 - userRepository.findByEmail(email)
 - NO existe → lanza RuntimeException("Credenciales incorrectas")
 - (mismo mensaje — no revelamos si el email existe)
- └─ 2. Compara la contraseña con el hash BCrypt

```
| passwordEncoder.matches(passwordTextoPlano, hashBD)
| NO coincide → lanza RuntimeException("Credenciales incorrectas")
|
|— 3. Genera el JWT
|
|— 4. Devuelve AuthResponse { token, user }
```

Frontend

```
| Guarda el token en localStorage
|
| \ \
```

Flujo de Petición Protegida (GET /auth/me)

\ \ \

Frontend

```
|
| GET /auth/me
| Header: Authorization: Bearer eyJ...
|
|
```



api-gateway

```
| JwtAuthFilter valida el token:
| - Verifica la firma con el JWT_SECRET
| - Comprueba que no ha expirado
| - Extrae el userId (subject del token)
| - Inyecta header X-User-Id: "uuid-del-usuario"
|
|
```



AuthController.me()

```
| Recibe X-User-Id del header — no toca el JWT directamente
|
|
```



AuthService.getUser(userId)

```
| userRepository.findById(userId)
| Hibernate: SELECT * FROM users WHERE id = ?
|
|— Devuelve UserEntity (sin contraseña — @JsonIgnore)
```

Frontend

└─ Muestra los datos en el dashboard

El token viaja en cada petición:

Authorization: Bearer eyJhbGciOiJIUzI1NiJ9...

El gateway lo verifica — si alguien lo modifica la firma no coincide y la petición se rechaza con 401 Unauthorized

...

Decisiones de diseño

| Decisión | Motivo |

|---|---|

| Sin Lombok | Ver la lógica explícita durante el desarrollo |

| `@JsonIgnore` en password | La contraseña nunca viaja en respuestas JSON (como `\$hidden` en Laravel) |

| Mismo mensaje en login | No revelar si el email existe o no (seguridad) |

| BCrypt con salt | Cada hash es diferente aunque la contraseña sea igual |

| JWT firmado con HMAC-SHA256 | Si alguien modifica el token, la firma no coincide |

| Rol USER por defecto | Nunca se puede autoasignar ADMIN desde el registro |

| Inyección por constructor | Evita dependencias circulares y facilita los tests |

| Gateway valida el JWT | Cada microservicio no necesita validar el token — lo hace el gateway |

Cómo probarlo (con curl)

Registrar usuario

` `` bash

curl -X POST http://localhost:8080/auth/register \

-H "Content-Type: application/json" \

-d '{

"email": "juan@spendiq.com",

"password": "password123",

"nombre": "Juan",

"apellido": "García",

```
"saldoInicial": 1000
}'
` ``
```

Login

```
` `` bash
curl -X POST http://localhost:8080/auth/login \
-H "Content-Type: application/json" \
-d '{
  "email": "juan@spendiq.com",
  "password": "password123"
}'
` ``
```

Ver perfil (con token)

```
` `` bash
curl http://localhost:8080/auth/me \
-H "Authorization: Bearer <TOKEN>"
` ``
```

Actualizar perfil

```
` `` bash
curl -X PUT http://localhost:8080/auth/me \
-H "Authorization: Bearer <TOKEN>" \
-H "Content-Type: application/json" \
-d '{
  "nombre": "Juan Carlos",
  "saldoInicial": 2000
}'
` ``
```